SEMINARIUM DYPLOMOWE 1 · MINI PW

# Funkcje sklejane i ich własności

od listwy szkutnika do sieci neuronowych

**Wojciech Jasiński** · 2 listopada 2026 · prowadzący: prof. Bohdan Macukow



# Idea funkcji sklejanej

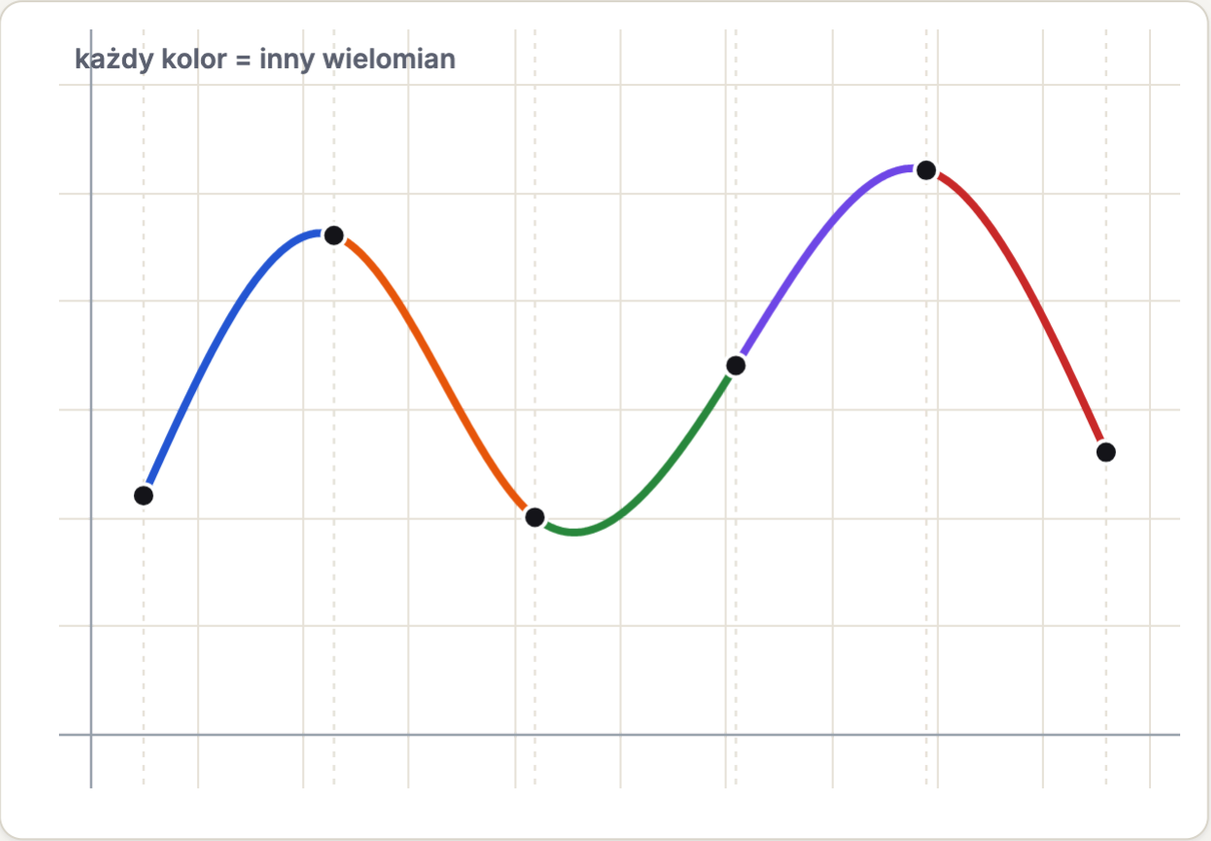


Mamy punkty, np. pomiary co godzinę. Szukamy gładkiej krzywej przez nie.

1. Punkty to $(x_i, y_i)$; liczby $x_i$ to **węzły**.
2. Między sąsiednimi węzłami bierzemy **osobny wielomian** niskiego stopnia (zwykle 3).
3. W węzłach kawałki **sklejamy gładko**: bez przerwy (wartość), bez załamania (nachylenie), bez szarpnięcia (krzywizna).

FORMALNIE

Funkcja sklejana stopnia $k$: na każdym przedziale $[x_{i-1}, x_i]$ wielomian stopnia $\leq k$, a całość ma ciągłe pochodne do rzędu $k-1$.



# Definicja formalna i wymiar przestrzeni

DODATEK

**FUNKCJA SKLEJANA STOPNIA $k$**

Niech $\Delta: \; a = x_0 < x_1 < \cdots < x_n = b$. Funkcja $s$ jest **funkcją sklejaną stopnia** $k$ o węzłach $\Delta$, jeśli

$$s \in C^{k-1}[a,b] \;\text{ oraz }\; s|_{[x_{i-1},\,x_i]} \in \Pi_k, \;\; i = 1, \ldots, n.$$

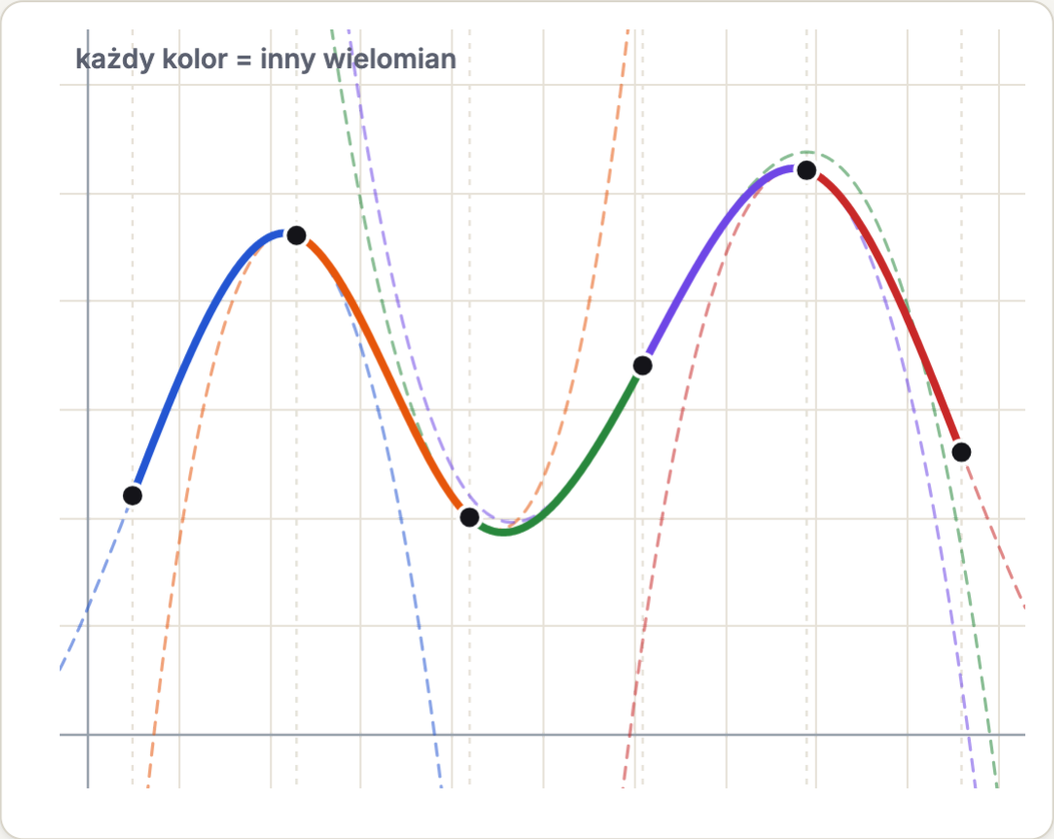
**Wymiar przestrzeni $S_k(\Delta)$:**

$$\dim S_k(\Delta) = \underbrace{n(k+1)}_{\text{współczynniki}} - \underbrace{(n-1)\,k}_{\text{warunki sklejenia}} = n + k.$$

**Baza potęg uciętych**, $(x-t)_+ = \max(0,\, x-t)$:

$$1,\; x,\; \ldots,\; x^k,\;\; (x-x_1)_+^k,\; \ldots,\; (x-x_{n-1})_+^k$$

$(x-t)_+$ to ReLU, stąd związek z sieciami neuronowymi.




stopień k 3 ciągłe: s, s′, s″

przedłuż kawałki

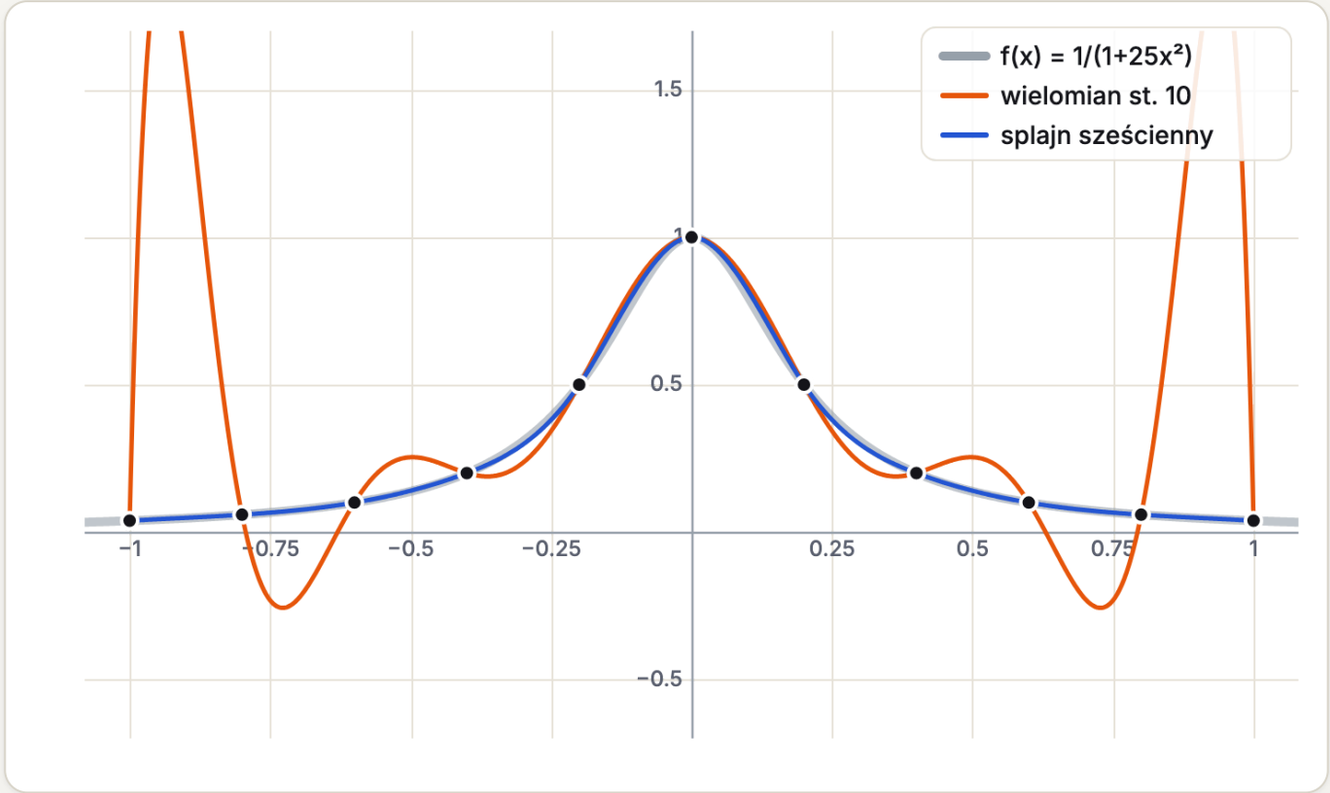
# Interpolacja wielomianowa i efekt Rungego



**Cel:** gładka funkcja przez $n+1$ danych punktów. Najprostszy pomysł: **jeden wielomian** stopnia $\leq n$ (taki wielomian jest dokładnie jeden).

- **Problem** (Runge, 1901)**:** dla $f(x) = 1/(1+25x^2)$ i punktów równoodległych wielomian oscyluje przy końcach, a błąd **rośnie** wraz z $n$.
- **Splajn** (niebieski) na tych samych punktach: każdy kawałek ma stopień 3, oscylacji nie ma, a błąd maleje.

**Wniosek:** oscylacje biorą się z wysokiego stopnia. Zamiast jednego wielomianu wysokiego stopnia bierzemy wiele wielomianów niskiego stopnia, każdy na swoim przedziale.




n 10 węzły Czebyszewa max|f − pₙ| 1.92

max|f − s| 0.0220

# Rząd zbieżności splajnu sześciennego

DODATEK

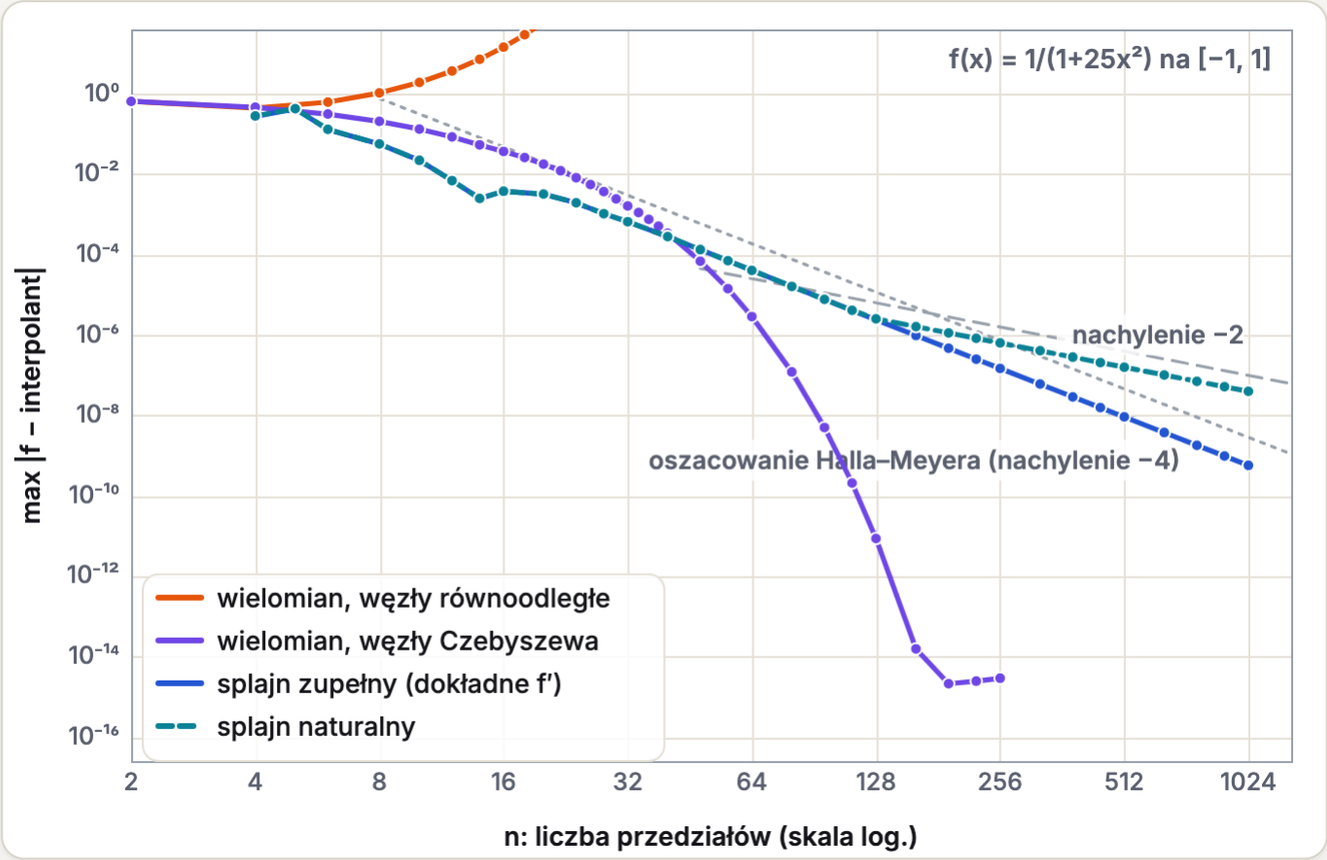
Hall i Meyer (1976)

$f \in C^4[a,b]$, $s$ to splajn zupełny z $s'(a) = f'(a)$, $s'(b) = f'(b)$; $h = \max h_i$:

$$\|f - s\|_\infty \le \tfrac{5}{384}\, h^4\, \|f^{(4)}\|_\infty$$
$$\|f' - s'\|_\infty \le \tfrac{1}{24}\, h^3\, \|f^{(4)}\|_\infty$$
$$\|f'' - s''\|_\infty \le \tfrac{3}{8}\, h^2\, \|f^{(4)}\|_\infty$$

Stałe $\frac{5}{384}$ i $\frac{1}{24}$ są optymalne.

- Warunki naturalne przy $f''(a) \neq 0$: błąd przy brzegu tylko $O(h^2)$.
- Dla funkcji analitycznych wielomian w węzłach Czebyszewa zbiega **wykładniczo**; splajny wygrywają lokalnością, stabilnością i odpornością na mało gładkie dane.




równoodległe Czebyszew zupełny naturalny

# Sześcienna funkcja sklejana interpolacyjna



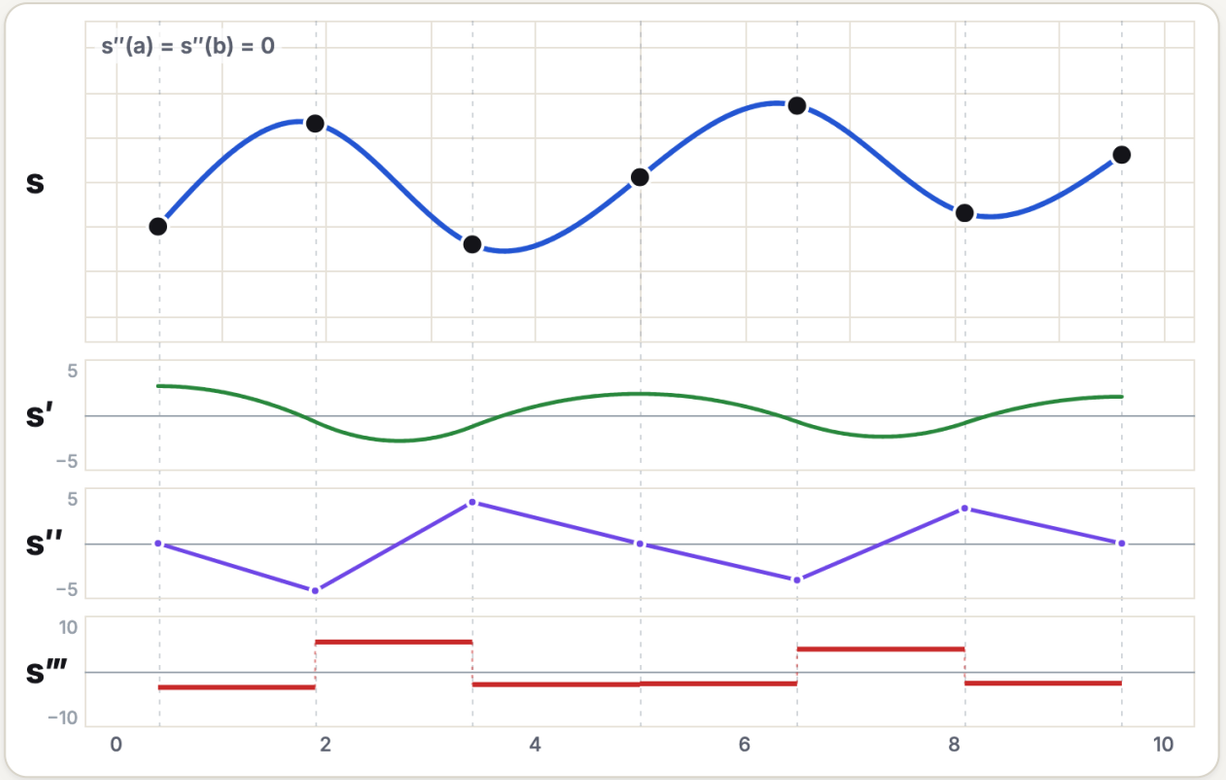
**Stopień 3** to najniższy, przy którym krzywizna ($s''$) jest ciągła.

**Bilans** niewiadomych i warunków dla $n$ kawałków:

| | |
|---|---|
| niewiadome: każdy kawałek ma 4 współczynniki | $4n$ |
| warunki: kawałek przechodzi przez swoje 2 punkty | $2n$ |
| warunki: ciągłe $s'$ i $s''$ w $n-1$ węzłach wewnątrz | $2n-2$ |
| **brakuje:** $4n-2n-(2n-2)$ | **2** |

Brakujące 2 warunki dodaje się **na końcach**:

- **naturalne:** $s''(a)=s''(b)=0$; na końcach krzywa się prostuje
- **zupełne:** zadane nachylenia $s'(a)$, $s'(b)$
- **not-a-knot:** dwa pierwsze kawałki to jeden wielomian, dwa ostatnie też (domyślne w SciPy)
- **okresowe:** na obu końcach te same $s'$ i $s''$; do krzywych zamkniętych




naturalne | zupełne | not-a-knot | okresowe  $\int s''^2\,dx$ 40.1 (minimum)

reset

# Wyznaczanie splajnu: układ równań liniowych



1. **Niewiadome:** krzywizny w węzłach $M_i = s''(x_i)$. Gdy je znamy, każdy kawałek wynika ze wzoru (dodatek ↓).
2. **Równania:** w każdym węźle wewnętrznym nachylenie z lewej równa się nachyleniu z prawej. Dla węzłów co $h$:

$$M_{i-1} + 4M_i + M_{i+1} = d_i,$$

$$d_i = \tfrac{6}{h^2}\,(y_{i-1} - 2y_i + y_{i+1}).$$

3. **Układ:** równania są **liniowe** w $M_i$, więc zbieramy je w jeden układ $A\,M = d$ (po prawej).

$$\underbrace{\begin{pmatrix} 4 & 1 & & & \\ 1 & 4 & 1 & & \\ & \ddots & \ddots & \ddots & \\ & & 1 & 4 & 1 \\ & & & 1 & 4 \end{pmatrix}}_{A} \begin{pmatrix} M_1 \\ M_2 \\ \vdots \\ M_{n-2} \\ M_{n-1} \end{pmatrix} = \begin{pmatrix} d_1 \\ d_2 \\ \vdots \\ d_{n-2} \\ d_{n-1} \end{pmatrix}$$

dane $y_i$ → $d$ → rozwiązanie $A\,M = d$ → $M_i$ → kawałki

- $A$ jest trójdiagonalna, przekątna dominuje ($4 > 1 + 1$) ⇒ **dokładnie jedno rozwiązanie**. Eliminacja Gaussa tylko na trzech przekątnych (algorytm Thomasa): $O(n)$ działań.
- **Liniowość:** $d$ zależy liniowo od danych $y_i$, więc cały splajn też: splajn dla sumy danych to suma splajnów.
- **Inne stopnie:** ten sam schemat; w bazie B-splajnów macierz jest pasmowa, koszt nadal $O(n)$.

# Układ trójdiagonalny: wzory i kod

DODATEK

Niewiadome: **momenty** $M_i = s''(x_i)$; $h_i = x_{i+1} - x_i$. Na $[x_i, x_{i+1}]$:

$$s(x) = M_i \frac{(x_{i+1} - x)^3}{6h_i} + M_{i+1} \frac{(x - x_i)^3}{6h_i} + \left(y_i - \frac{M_i h_i^2}{6}\right) \frac{x_{i+1} - x}{h_i} + \left(y_{i+1} - \frac{M_{i+1} h_i^2}{6}\right) \frac{x - x_i}{h_i}$$

Ciągłość $s'$ w $x_i$, $i = 1, \ldots, n-1$:

$$h_{i-1} M_{i-1} + 2(h_{i-1} + h_i)\, M_i + h_i M_{i+1} = 6 \left( \frac{y_{i+1} - y_i}{h_i} - \frac{y_i - y_{i-1}}{h_{i-1}} \right)$$

- Warunki naturalne: $M_0 = M_n = 0$; macierz dla $M_1, \ldots, M_{n-1}$ jest symetryczna i **ściśle diagonalnie dominująca** ⇒ rozwiązanie istnieje i jest jedyne.
- **Algorytm Thomasa:** $O(n)$ działań, stabilny bez wyboru elementu głównego.

```python
def thomas(a, b, c, d):
    """a: pod, b: na, c: nad przekątną"""
    n = len(b); c, d = c.copy(), d.copy()
    c[0] /= b[0]; d[0] /= b[0]
    for i in range(1, n):          # eliminacja
        m = b[i] - a[i] * c[i-1]
        c[i] = c[i] / m if i < n-1 else 0
        d[i] = (d[i] - a[i] * d[i-1]) / m
    for i in range(n-2, -1, -1): # wstecz
        d[i] -= c[i] * d[i+1]
    return d                       # M_0 … M_n
```

# Listwa kreślarska i twierdzenie Holladaya



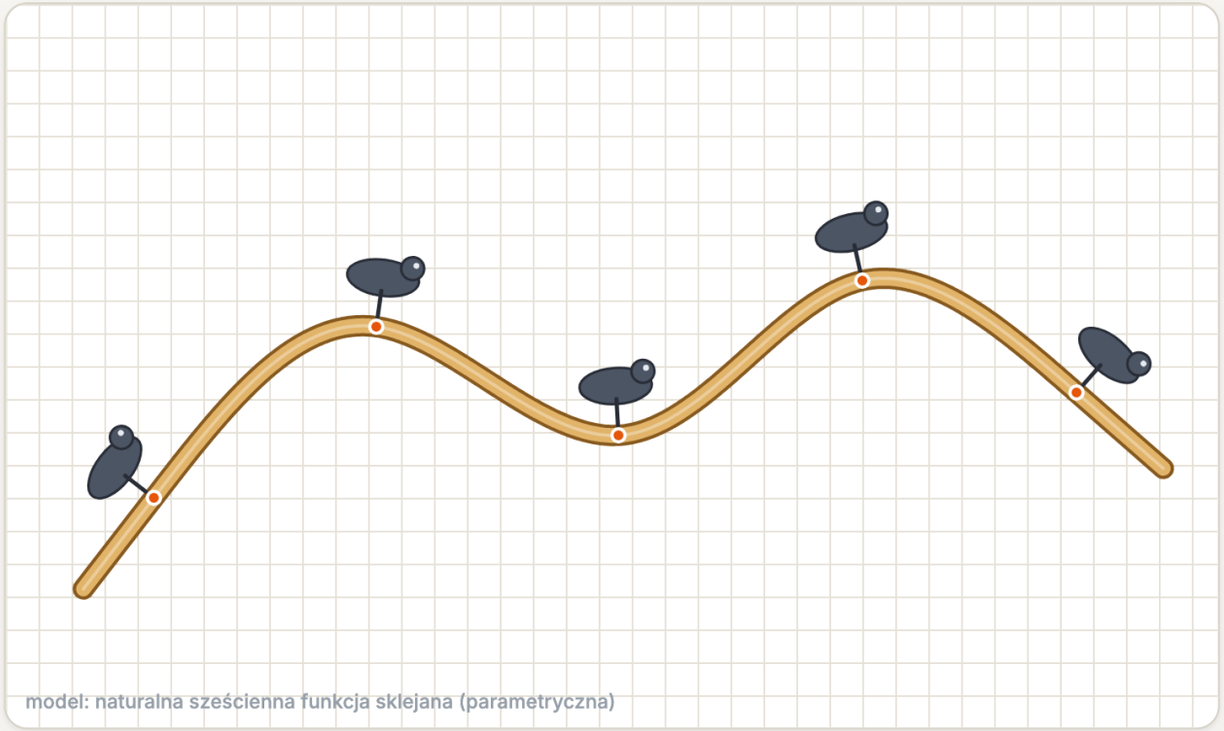
**spline** (ang.): giętka listwa kreślarska dociskana ołowianymi ciężarkami, tzw. **„kaczkami”**.



Wykreślanie w skali 1:1, zakłady lotnicze Rohr, Chula Vista, 1956 (San Diego Air & Space Museum, domena publiczna)

**TWIERDZENIE** (Holladay, 1957)

Spośród wszystkich funkcji klasy $C^2$ przez dane punkty najmniejszą $\int f''^2$ ma **naturalny splajn sześcienny**.




grzebień krzywizny | reset | energia zginania $\int \kappa^2\,ds$ 46.02 $(\times 10^{-3}\ px^{-1})$

To energia zginania listwy, więc listwa układa się w splajn. Za ostatnią kaczką jest prosta: $s'' = 0$, **warunek naturalny**.

# Twierdzenie Holladaya i fizyka listwy

DODATEK

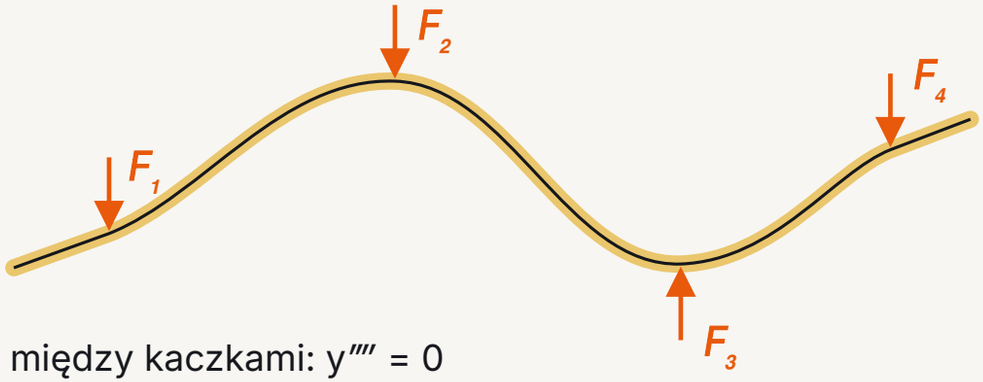
**TWIERDZENIE** (Holladay, 1957)

Spośród **wszystkich** funkcji klasy $C^2$ przechodzących przez dane punkty najmniejszą „energię zginania" ma **naturalny splajn sześcienny**:

$$\int_a^b s''(x)^2\,dx \;\le\; \int_a^b f''(x)^2\,dx.$$

- Miarą niegładkości jest całka z kwadratu drugiej pochodnej.
- Dowód: całkowanie przez części (dodatek ↓).

## Fizyka listwy




między kaczkami: $y'''' = 0$

- Energia zginania listwy (małe ugięcia): $\frac{EI}{2}\int y''^2\,dx$, czyli **ta sama całka**.
- Między kaczkami nie działa żadna siła $\Rightarrow y'''' = 0 \Rightarrow$ **wielomian stopnia 3**.
- Swobodny koniec $\Rightarrow y'' = 0$, czyli **warunek naturalny**.

# Dowód twierdzenia Holladaya



Niech $s$ będzie naturalną sześcienną funkcją sklejaną, $s(x_i) = y_i$, i niech $f \in C^2[a,b]$, $f(x_i) = y_i$. Oznaczmy $e = f - s$; wtedy $e(x_i) = 0$ i

$$\int_a^b f''^2 = \int_a^b s''^2 + \int_a^b e''^2 + 2\int_a^b s''e''.$$

Wyraz mieszany znika; całkujemy przez części ($s'''$ jest stała, $= c_i$, na każdym przedziale):

$$\int_a^b s''e'' = \big[s''e'\big]_a^b - \sum_i c_i \int_{x_i}^{x_{i+1}} e' = 0 - \sum_i c_i \big(e(x_{i+1}) - e(x_i)\big) = 0,$$

bo $s''(a) = s''(b) = 0$ (warunki naturalne) i $e$ zeruje się w węzłach. Zatem

$$\int f''^2 = \int s''^2 + \int e''^2 \;\geq\; \int s''^2,$$

a równość zachodzi tylko, gdy $e'' \equiv 0$, czyli $e$ jest liniowa i zeruje się w węzłach $\Rightarrow f = s$. ■

Model listwy jest zlinearyzowany: dla małych ugięć krzywizna $\kappa \approx y''$. Pełny problem nieliniowy to krzywa sprężysta (elastica) Eulera.

# Rys historyczny

DODATEK

matematyka przemysł informatyka i ML



**1691–1744**
**J. Bernoulli, L. Euler**
krzywa sprężysta (elastica), matematyka giętkiej listwy

**1901**
**C. Runge**
interpolacja wielomianem może się rozbiegać

**1912**
**S. Bernstein**
wielomiany Bernsteina: konstruktywny dowód tw. Weierstrassa

**1946**
**I. J. Schoenberg**
nazwa spline functions; wcześniej: tablice balistyczne w Aberdeen

**1957**
**J. C. Holladay**
splajn naturalny minimalizuje $\int f''^2$

**1959–1962**
**P. de Casteljau, P. Bézier**
Citroën i Renault: krzywe do projektowania karoserii

**1960–1964**
**C. de Boor; J. Ferguson**
General Motors i Boeing: krzywe i powierzchnie sklejane

**1967–1970**
**C. Reinsch; G. Kimeldorf, G. Wahba**
splajny wygładzające ↔ estymacja bayesowska

**1972**
**M. Cox, C. de Boor**
stabilna rekurencja dla B-splajnów

**1974–1978**
**Catmull–Rom, Chaikin, Versprille, Catmull–Clark**
grafika komputerowa, NURBS (1975), podział powierzchni

**1984 / 1991**
**PostScript, TrueType**
czcionki jako krzywe 3. i 2. stopnia

**2005–2024**
**IGA; sieci neuronowe; KAN**
MES na splajnach, „sieć = splajn", sieci Kołmogorowa–Arnolda

# B-splajny jako baza funkcji sklejanych



Każdą funkcję sklejaną stopnia $k$ zapisujemy jako $s(x) = \sum_i c_i \, B_{i,k}(x)$.

**DEFINICJA** (Cox, de Boor 1972), **WĘZŁY** $t_i$

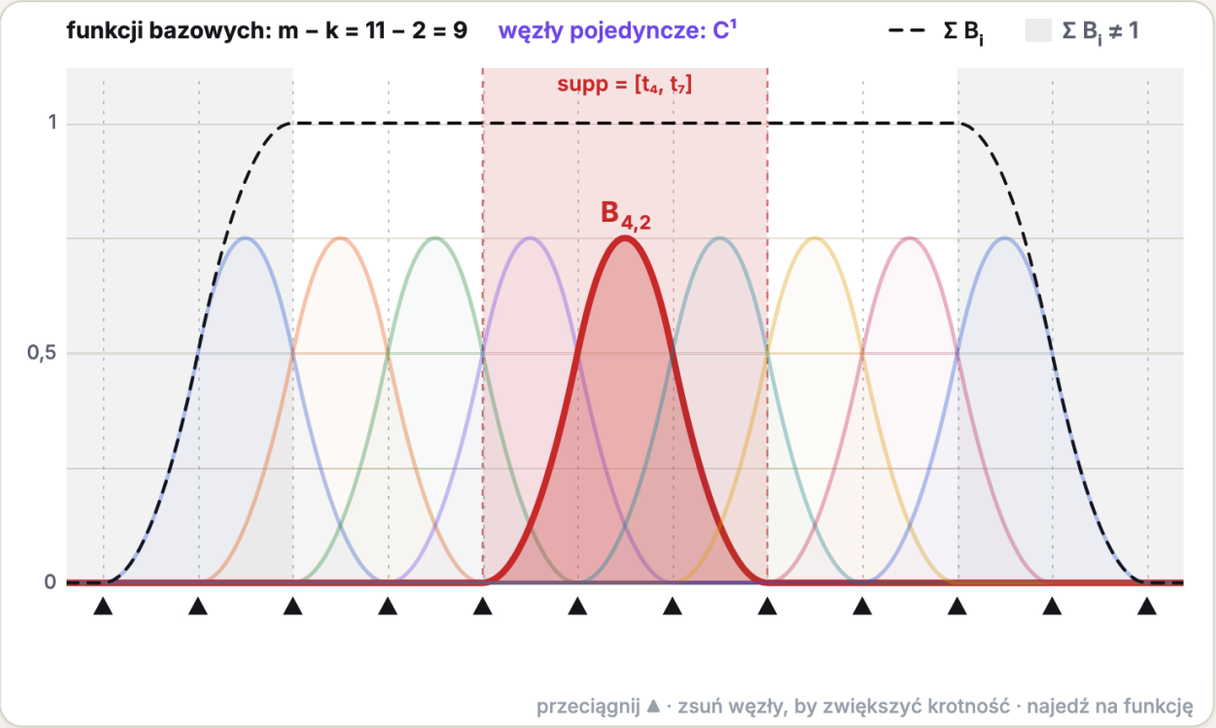
$B_{i,0} = 1$ na $[t_i, t_{i+1})$, poza nim $0$;

$$B_{i,k} = \omega_{i,k} \, B_{i,k-1} + \big(1 - \omega_{i+1,k}\big) \, B_{i+1,k-1},$$

gdzie $\omega_{i,k}(x) = \dfrac{x - t_i}{t_{i+k} - t_i}$. Każdy stopień to średnia ważona dwóch sąsiednich funkcji niższego stopnia.

- $B_{i,k}$ jest niezerowa tylko na $k+1$ sąsiednich przedziałach; funkcje są nieujemne i sumują się do 1.

**Po co:** zmiana jednego $c_i$ zmienia krzywą tylko lokalnie, a układy równań są pasmowe, więc szybkie. Dlatego to standardowy zapis splajnów w CAD i bibliotekach numerycznych.




stopień k 2

węzły | równomierne | otwarte (clamped) | losowe

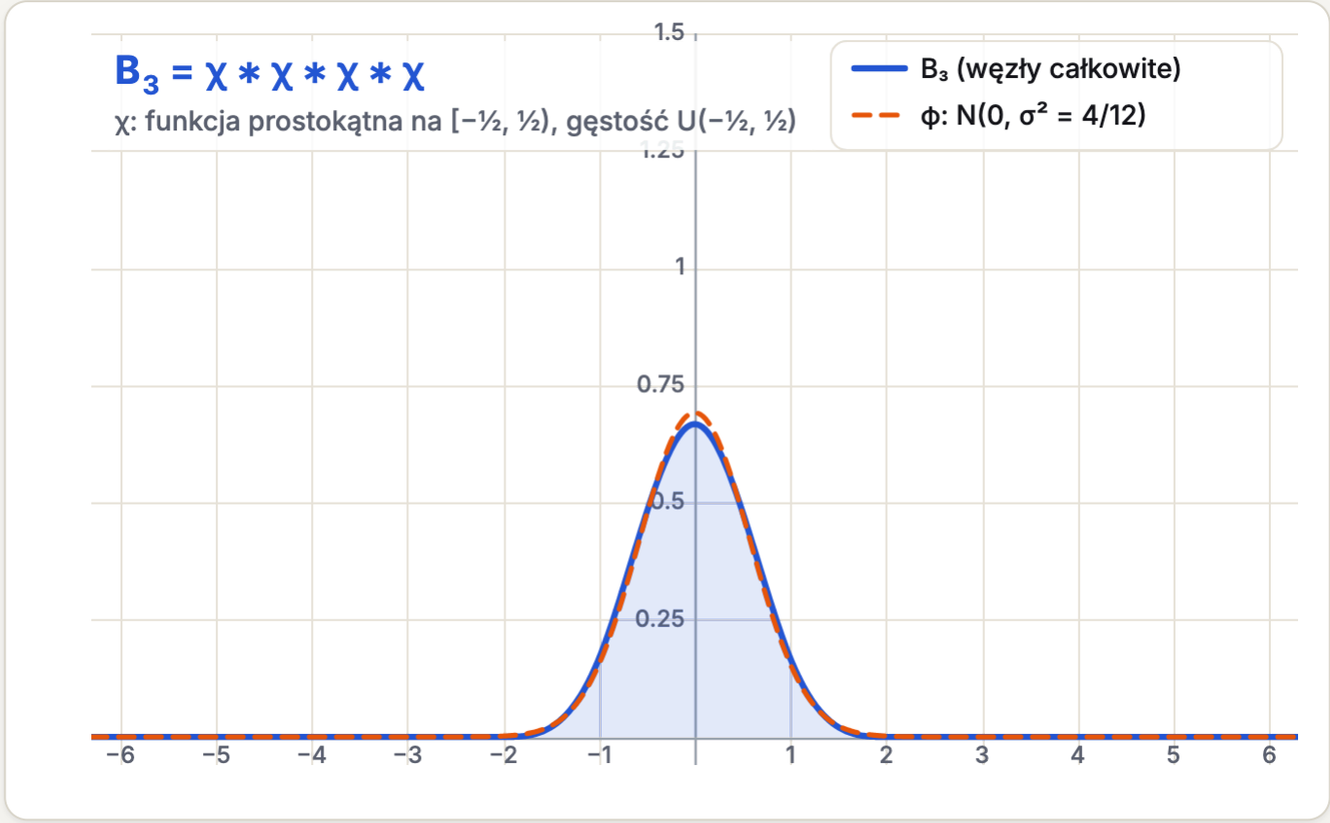
# B-splajny jako sploty i rozkład normalny

DODATEK

Dla węzłów równoodległych (całkowitych):

$$B_k = \underbrace{\chi_{[0,1)} * \chi_{[0,1)} * \cdots * \chi_{[0,1)}}_{k+1 \text{ razy}}$$

- $B_k$ = gęstość sumy $k+1$ niezależnych zmiennych $U[0,1]$ (rozkład Irwina–Halla).
- Średnia $\frac{k+1}{2}$, wariancja $\frac{k+1}{12}$; z CTG po standaryzacji gęstość dąży do gęstości $\mathcal{N}(0,1)$.
- Transformata Fouriera: $\widehat{B_k}(\omega) = \left(\frac{\sin(\omega/2)}{\omega/2}\right)^{k+1} e^{-i\omega(k+1)/2}$.
- Dlatego B-splajny dobrze sprawdzają się w przetwarzaniu sygnałów (Unser, 1999).




stopień k 3

pokaż niższe stopnie

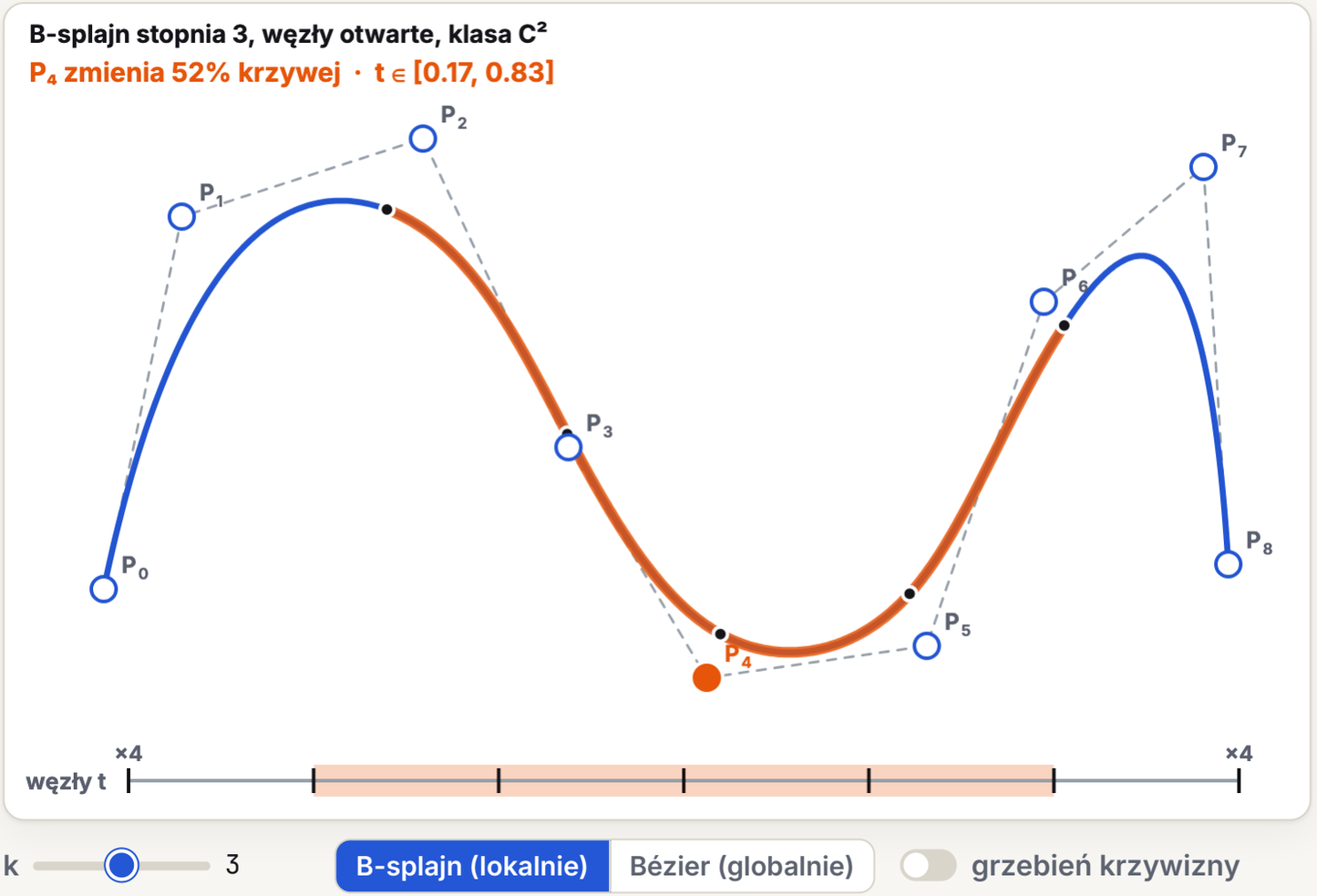
max |Bk − ϕ| 0.0243

# Krzywe Béziera i B-splajnowe



Krzywą sterują **punkty kontrolne**: $C(t) = \sum_i w_i(t)\, P_i$.

- **Bézier**: wagi to jeden wielomian na całej krzywej ⇒ każdy punkt zmienia **całą** krzywą. Takie krzywe rysuje „Pióro” w Figmie i Illustratorze.
- **B-splajn**: wagami są B-splajny ⇒ przesunięcie $P_i$ zmienia tylko **fragment**.
- W CAD płynność krzywizny sprawdza się **grzebieniem krzywizny**.



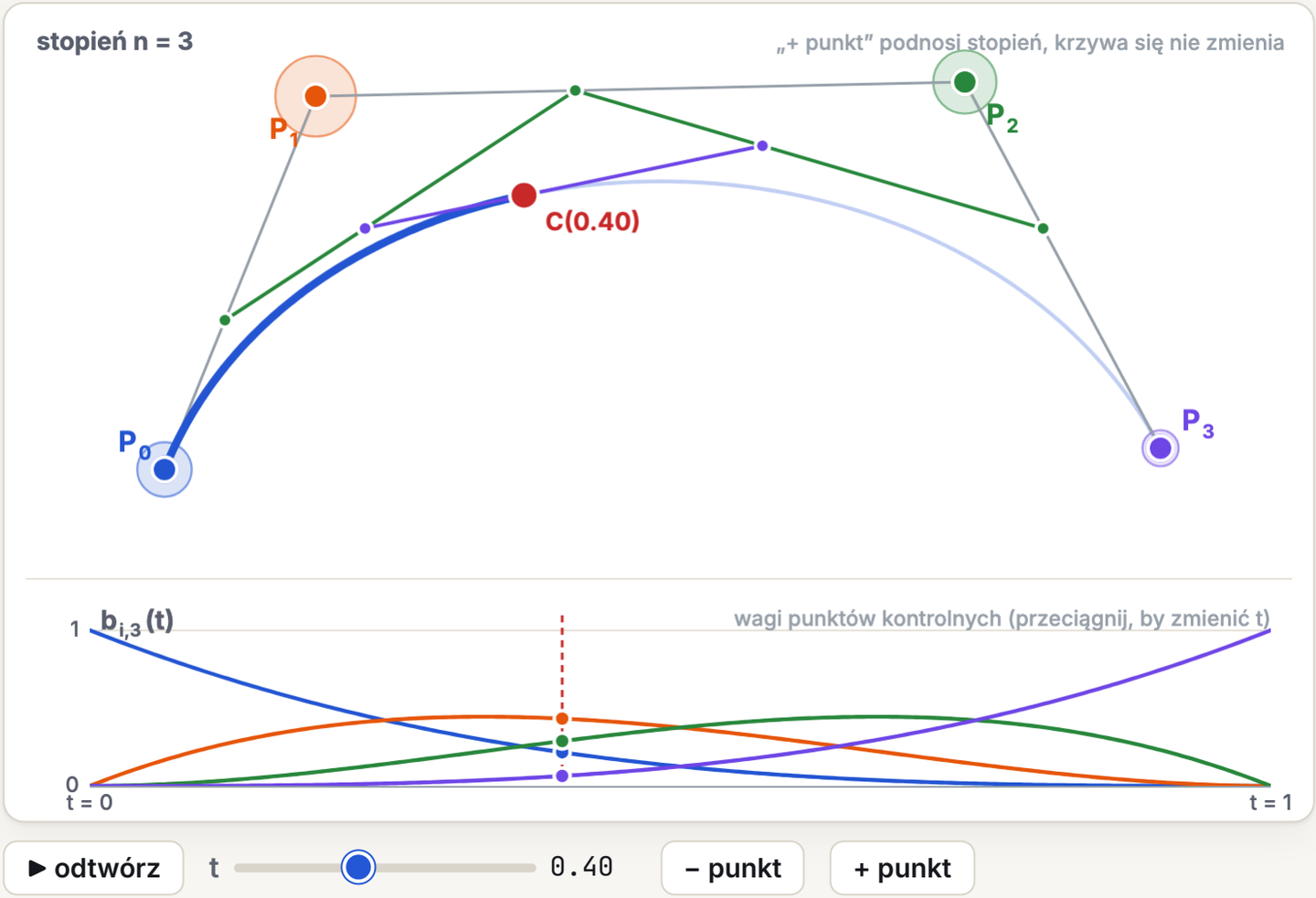
# Krzywe Béziera i algorytm de Casteljau

DODATEK

Krzywą wyznaczają **punkty kontrolne** $P_0, \ldots, P_n$, które ją „przyciągają", ale krzywa nie musi przez nie przechodzić:

$$C(t) = \sum_{i=0}^{n} \binom{n}{i} t^i (1-t)^{n-i} P_i, \quad t \in [0,1]$$

- **de Casteljau** (Citroën, 1959): punkt krzywej to powtarzane **dzielenie odcinków w proporcji** $t$. Prosty i stabilny algorytm.
- Zaczyna się w $P_0$, kończy w $P_n$, leży w otoczce wypukłej punktów.
- Wada: **sterowanie globalne**, każdy punkt zmienia całą krzywą.

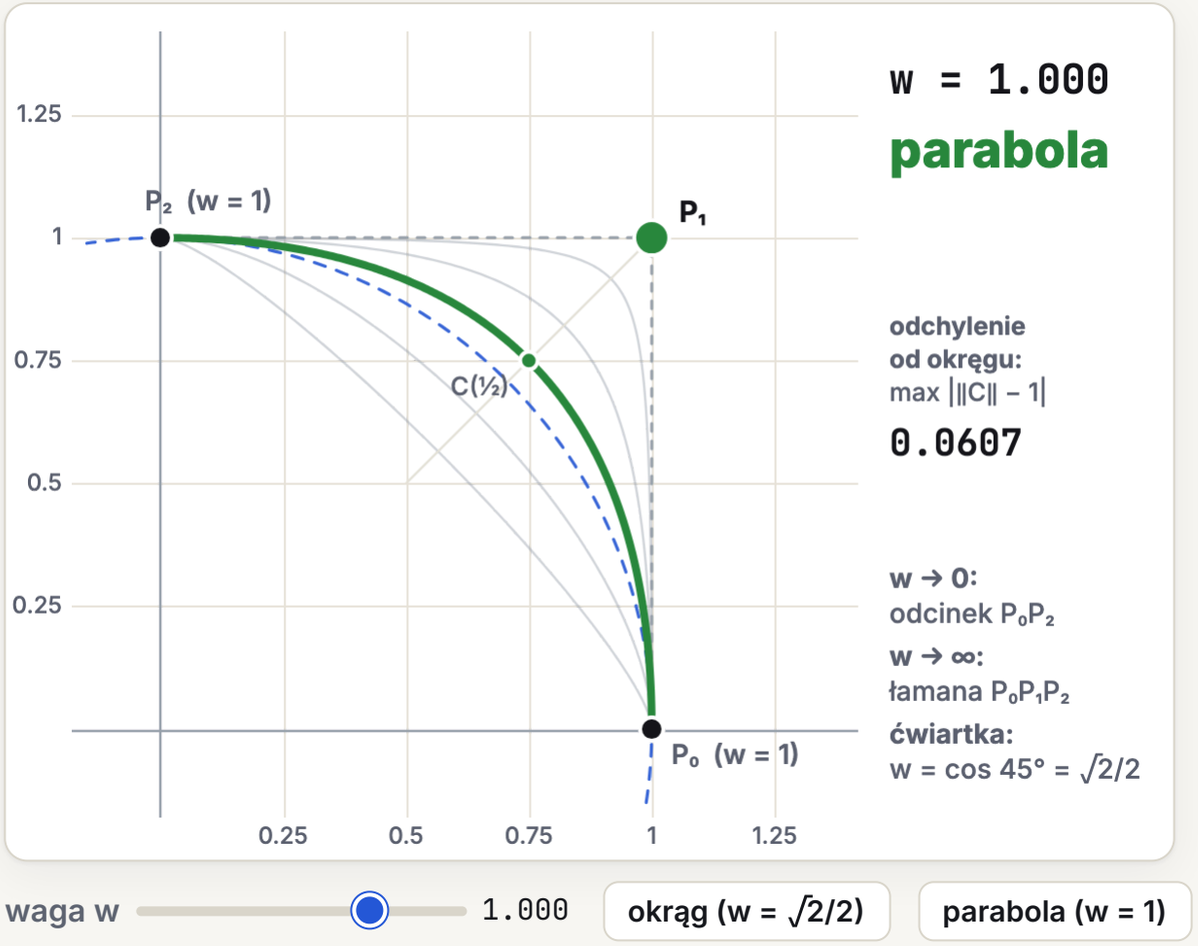


# NURBS: krzywe wymierne

DODATEK

$$C(t) = \frac{\sum_i w_i \, B_{i,k}(t) \, P_i}{\sum_i w_i \, B_{i,k}(t)}$$

- Wielomianem **nie da się** dokładnie narysować okręgu; krzywą wymierną można.
- Wymierna krzywa Béziera 2. stopnia, wagi końcowe 1, środkowa $w > 0$: $w < 1$ **elipsa**, $w = 1$ **parabola**, $w > 1$ **hiperbola**.
- Łuk o kącie $\theta$: $w = \cos(\theta/2)$; ćwiartka okręgu: $w = \frac{\sqrt{2}}{2}$.
- Dlatego standardy CAD (IGES, STEP) i silniki geometrii opierają się na NURBS (Versprille, 1975).

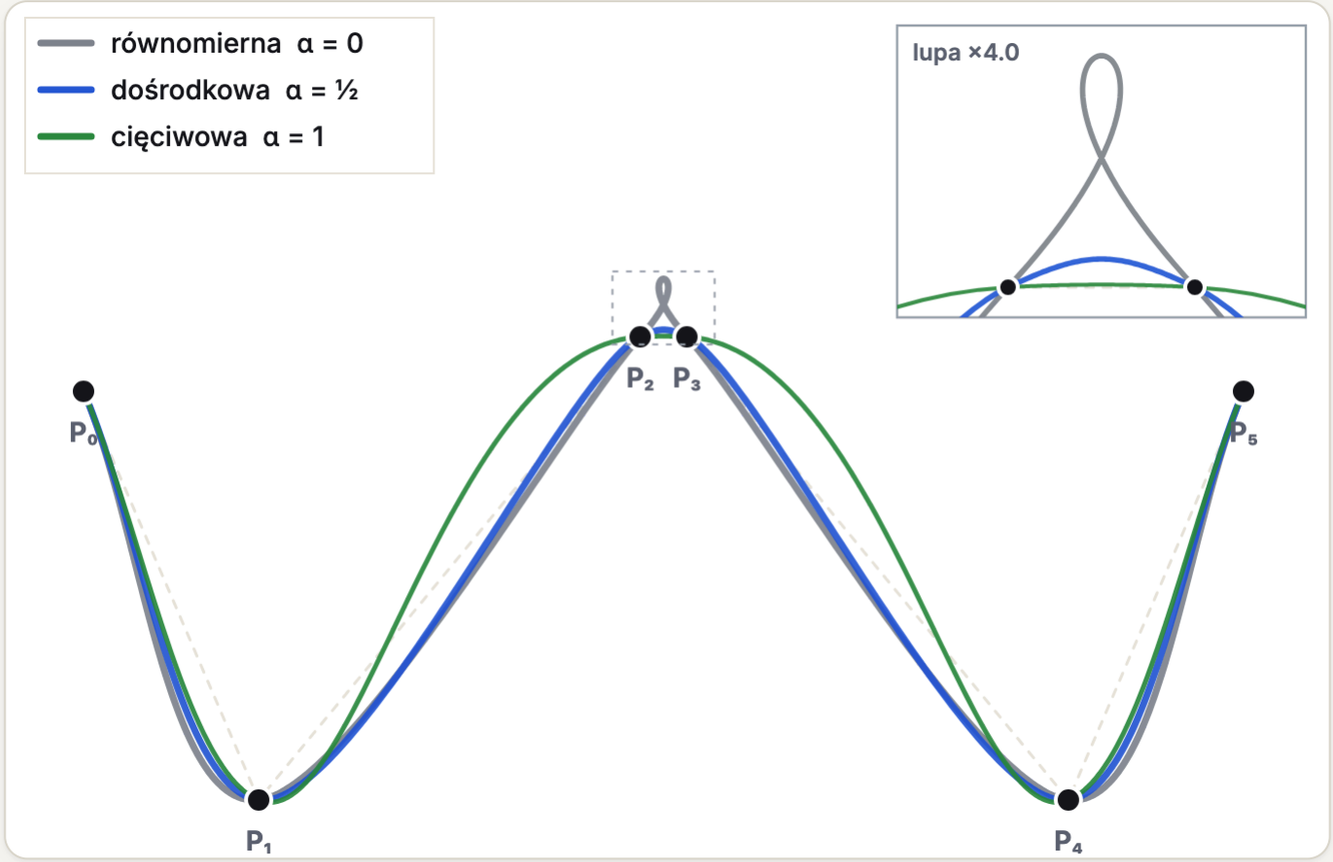


# Catmull–Rom: interpolacja i parametryzacja

DODATEK

- Przechodzi **przez** punkty, lokalna, klasy $C^1$ (Catmull, Rom 1974); standard w animacji i grach.
- Węzły $t_{i+1} = t_i + |P_{i+1} - P_i|^{\alpha}$: równomierna $\alpha = 0$, **dośrodkowa** $\alpha = \frac{1}{2}$, **cięciwowa** $\alpha = 1$.
- Wersja dośrodkowa nie ma ostrzy ani pętli wewnątrz segmentu (Yuksel i in., 2011).

Przeciągnij punkty blisko siebie: wersja równomierna robi pętle.



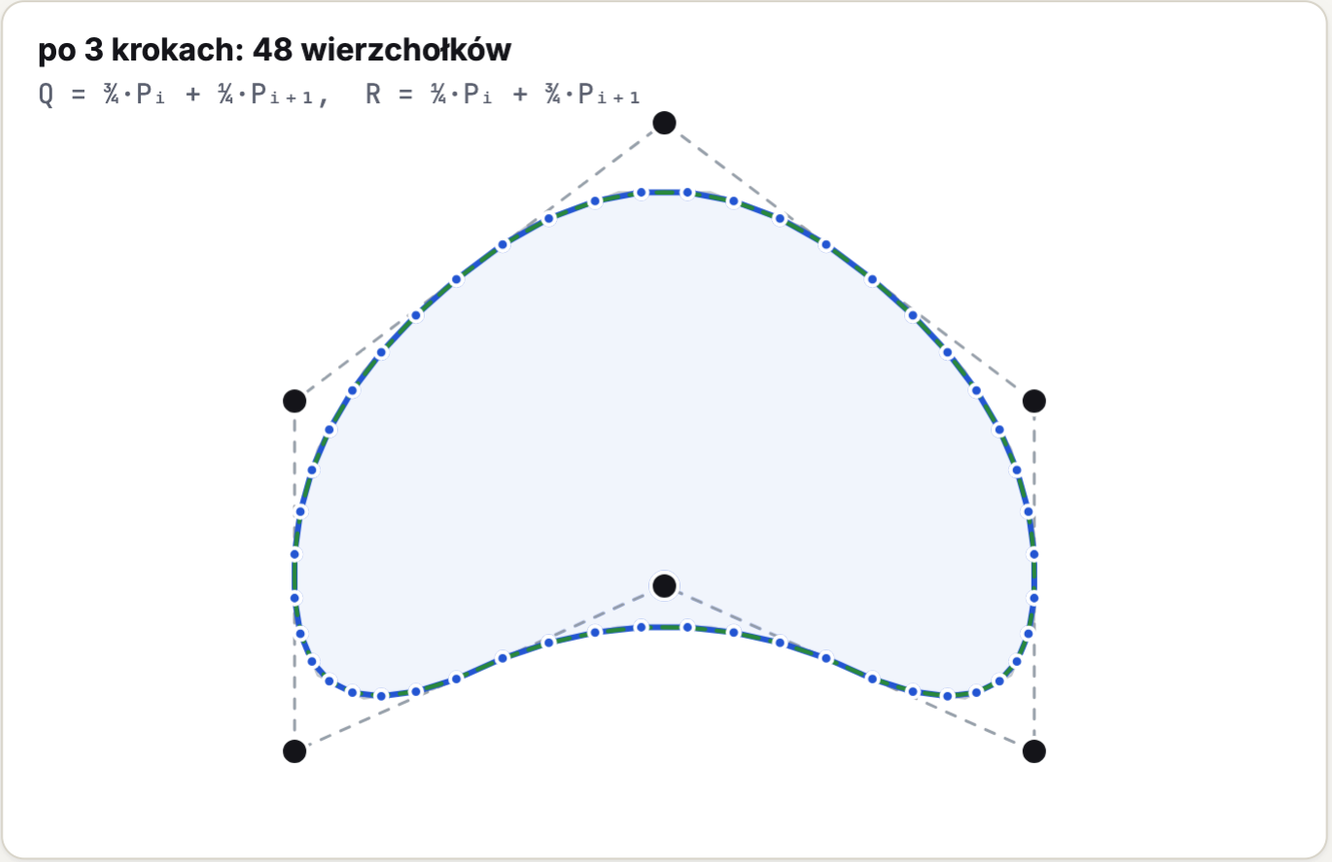
# Algorytm Chaikina

DODATEK

**Chaikin** (1974)**:** każdy odcinek $P_iP_{i+1}$ zastąp punktami

$$Q_i = \tfrac{3}{4}P_i + \tfrac{1}{4}P_{i+1}, \qquad R_i = \tfrac{1}{4}P_i + \tfrac{3}{4}P_{i+1}.$$

- Granica: **kwadratowa krzywa B-splajnowa** (Riesenfeld, 1975).
- Uogólnienie na powierzchnie: **Catmull–Clark** (1978), standard w animacji 3D; Pixar, Geri's Game (1997).



# Zastosowania





## Czcionki

Kształt litery zapisany krzywymi, nie pikselami, więc jest ostry w każdym rozmiarze.

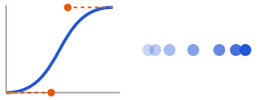
**Przykład:** formaty czcionek TrueType i OpenType (w każdym komputerze i telefonie) zapisują kontury krzywymi Béziera.



## CAD: samochody i samoloty

Gładkie powierzchnie karoserii, skrzydeł i kadłubów opisuje się splajnami (NURBS).

**Przykład:** model z programu CATIA lub SolidWorks zapisany w formacie STEP trafia w tej postaci do symulacji i na obrabiarkę CNC.



## Animacja i gry

Animator ustawia kilka klatek kluczowych, a ruch pomiędzy nimi program wylicza jako krzywą.

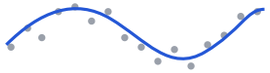
**Przykład:** krzywe animacji w programie Blender; ścieżki kamer i obiektów w silniku gier Unity (pakiet Splines).



## Robotyka i pojazdy

Trajektoria jako splajn: położenie, prędkość i przyspieszenie zmieniają się płynnie, bez szarpnięć.

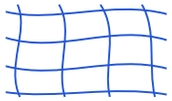
**Przykład:** typ ruchu SPLINE w sterownikach robotów przemysłowych firmy KUKA; wygładzanie toru jazdy w otwartym systemie autonomicznym Baidu Apollo.



## Statystyka i ML

Gdy zależność w danych nie jest prostą linią, dopasowuje się do niej splajn zamiast gotowego wzoru.

**Przykład:** `SplineTransformer` w bibliotece scikit-learn (Python); modele GAM, np. pakiet mgcv w języku R.



## Obrazowanie medyczne

Żeby porównać dwa skany tego samego pacjenta, jeden obraz odkształca się tak, by pasował do drugiego.

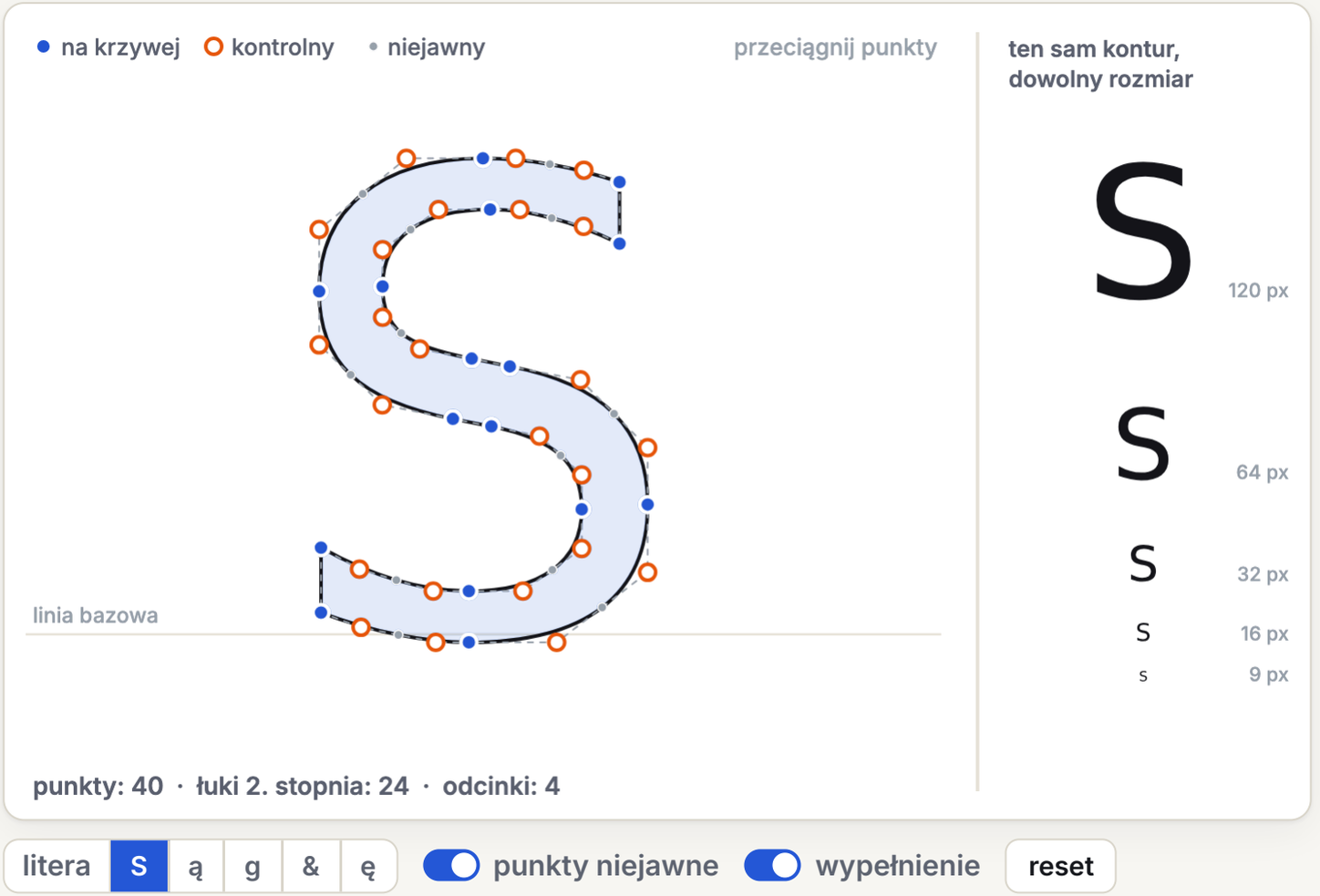
**Przykład:** odkształcenie opisuje siatka B-splajnów (Rueckert i in. 1999), np. w otwartym programie elastix do skanów CT i MRI.

# Czcionki: kontury TrueType



- **TrueType** (Apple, 1991): kontur = kwadratowe krzywe Béziera.
- Punkty **na krzywej** ● i **poza nią** ○ (kontrolne).
- **PostScript Type 1** (1984), dziś **CFF** w OpenType: krzywe 3. stopnia.
- Skalowanie bez utraty jakości: przekształcamy tylko punkty kontrolne.

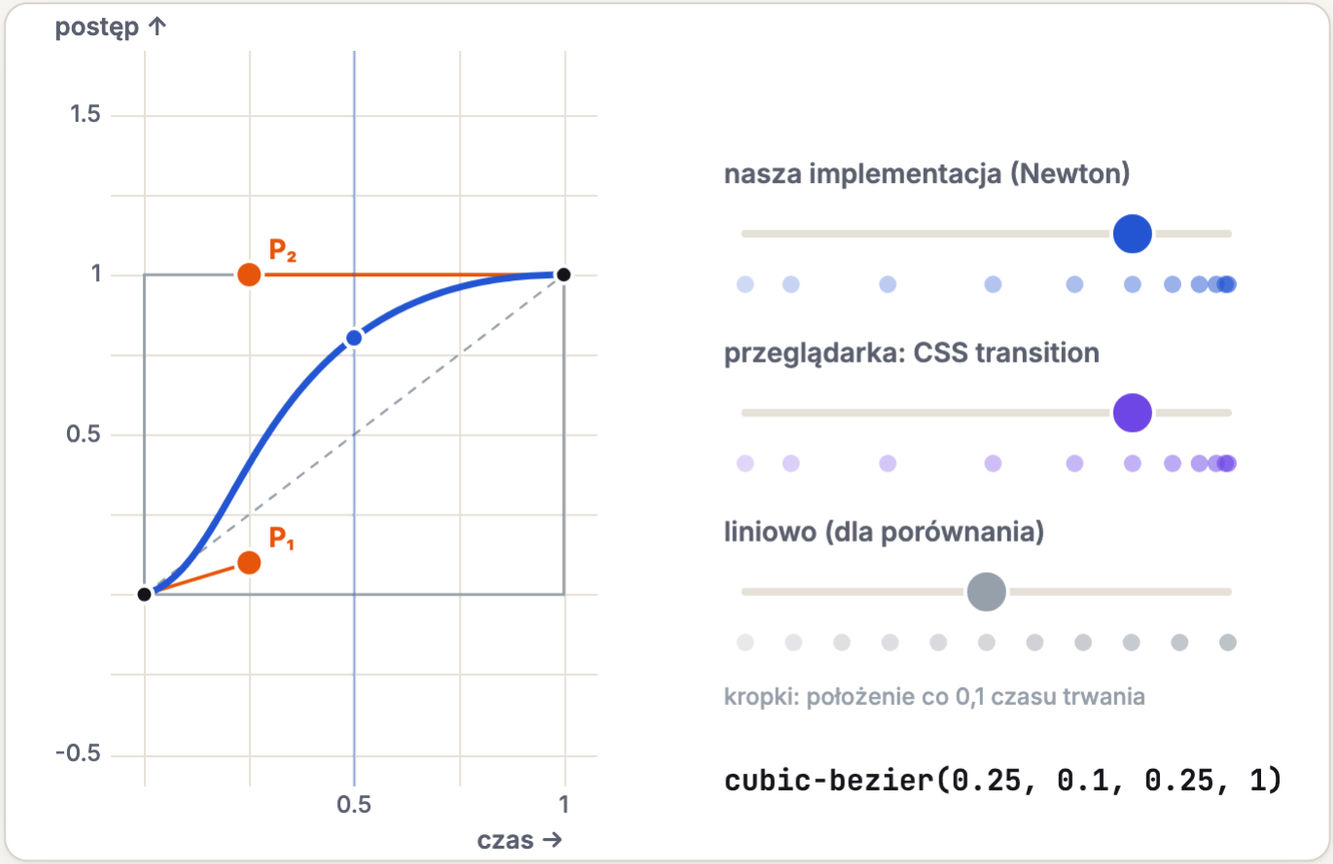
Kontury: DejaVu Sans (wolna licencja).



# Animacje w CSS: `cubic-bezier()`

DODATEK

- CSS: krzywa Béziera 3. stopnia z $P_0 = (0, 0)$, $P_3 = (1, 1)$; użytkownik wybiera $P_1$, $P_2$.
- Oś $x$ = czas, oś $y$ = postęp animacji.
- Problem: mamy czas $x$, potrzebujemy $y \Rightarrow$ trzeba odwrócić $x(t)$ **metodą Newtona** (z bisekcją jako asekuracją), tak robią przeglądarki.
- $y \notin [0, 1] \Rightarrow$ efekt „sprężyny".



# Jak komputer przechowuje i liczy splajny

DODATEK

Ten sam splajn można zapisać na dwa sposoby; podstawowe operacje są tanie ($n$ = liczba punktów).

DWA SPOSOBY ZAPISU

**Kawałkami:** dla każdego przedziału współczynniki jego wielomianu. Wartość liczy się najszybciej.

**W bazie B-splajnów:** węzły i współczynniki $c_i$. Obliczenia stabilne, zmiany lokalne; tak zapisują splajny programy CAD i biblioteki (np. SciPy).

| OPERACJA | JAK | KOSZT |
|---|---|---|
| wyznaczenie splajnu przez $n$ punktów | układ trójdiagonalny | $O(n)$ |
| wartość $s(x)$ w punkcie | znaleźć przedział, policzyć wielomian | $O(\log n)$ |
| wygładzenie $n$ pomiarów | układ pasmowy (Reinsch) | $O(n)$ |
| narysowanie krzywej | podział na $m$ krótkich odcinków | $O(m)$ |

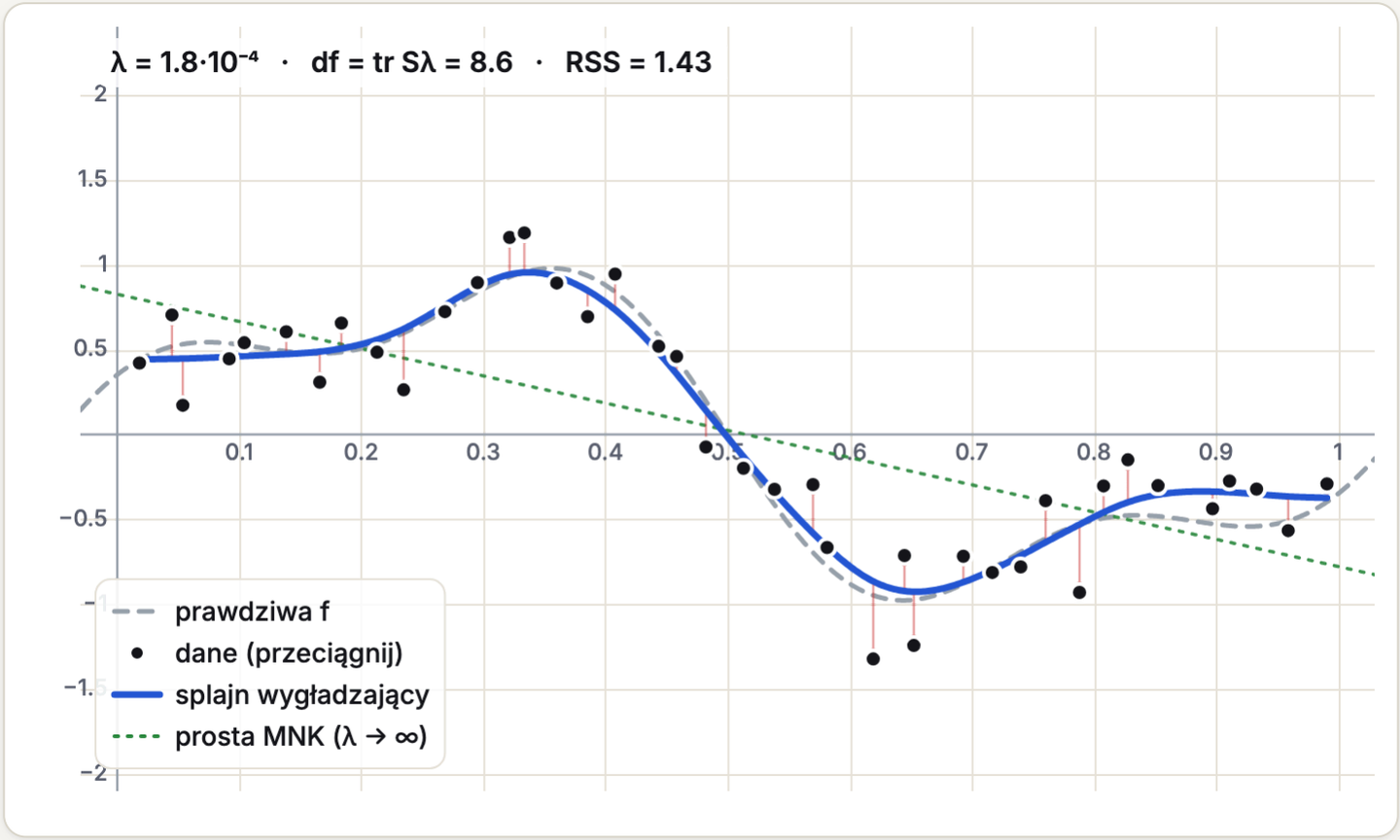
# Analiza danych: splajn wygładzający



Pomiary mają szum, więc nie chcemy przechodzić **przez** punkty, tylko **blisko** nich:

$$\min_f \underbrace{\sum_i \big(y_i - f(x_i)\big)^2}_{\text{dopasowanie}} + \lambda \underbrace{\int f''(x)^2\,dx}_{\text{krzywizna}}$$

- Rozwiązanie to znowu **naturalny splajn sześcienny** z węzłami w $x_i$ (Reinsch, 1967). To wniosek z tw. Holladaya.
- $\lambda \to 0$: interpolacja (przeuczenie); $\lambda \to \infty$: prosta regresji.
- $\lambda$ dobiera się automatycznie z danych (walidacja krzyżowa, GCV).

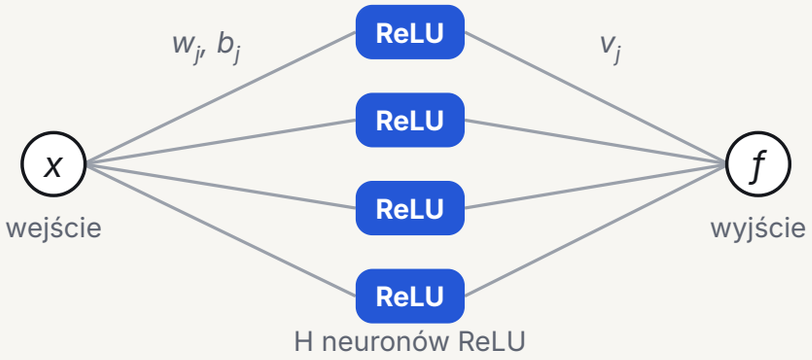



λ 1.8·10-4 interpolacja λ z GCV prosta nowe dane

# Sieć neuronowa z ReLU to funkcja sklejana

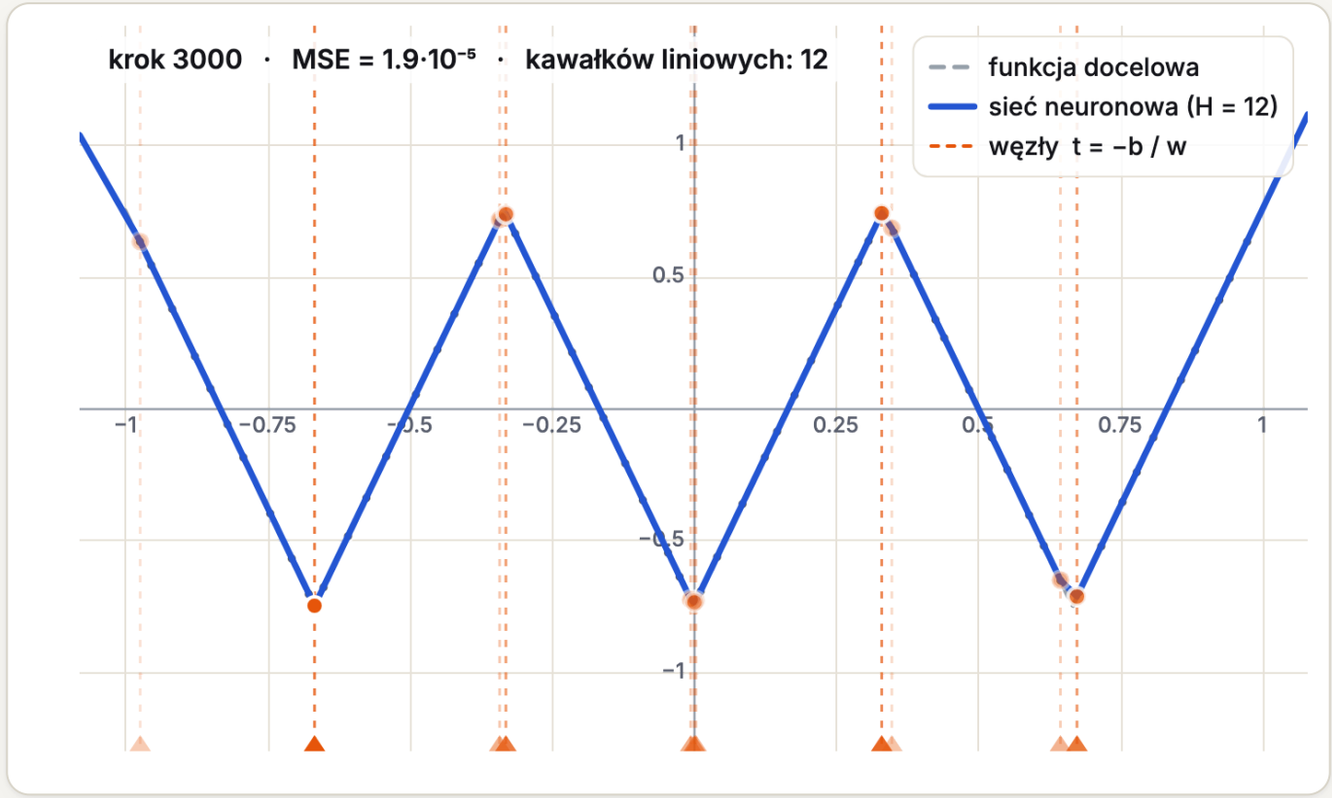


Sieć z jedną warstwą ukrytą, $\mathrm{ReLU}(x) = \max(0,\, x)$:




$$f(x) = c + \sum_{j=1}^{H} v_j \ \mathrm{ReLU}(w_j x + b_j)$$

- Każdy neuron dodaje jedno załamanie w $t_j = -b_j / w_j \Rightarrow$ wynik to **łamana**, czyli splajn liniowy.
- Różnica: **węzły też są uczone**.
- Więcej warstw: nadal kawałkami liniowa (Balestriero, Baraniuk 2018).

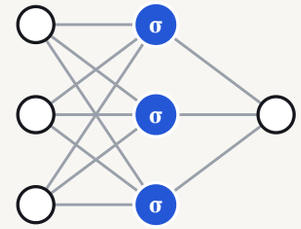


# KAN: sieci Kołmogorowa–Arnolda (2024)

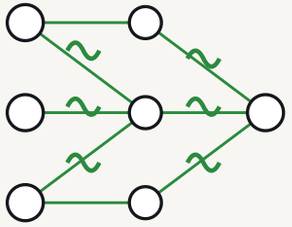
DODATEK

**Zwykła sieć (MLP):** na krawędziach liczby (wagi), w neuronach ta sama stała funkcja, np. ReLU.

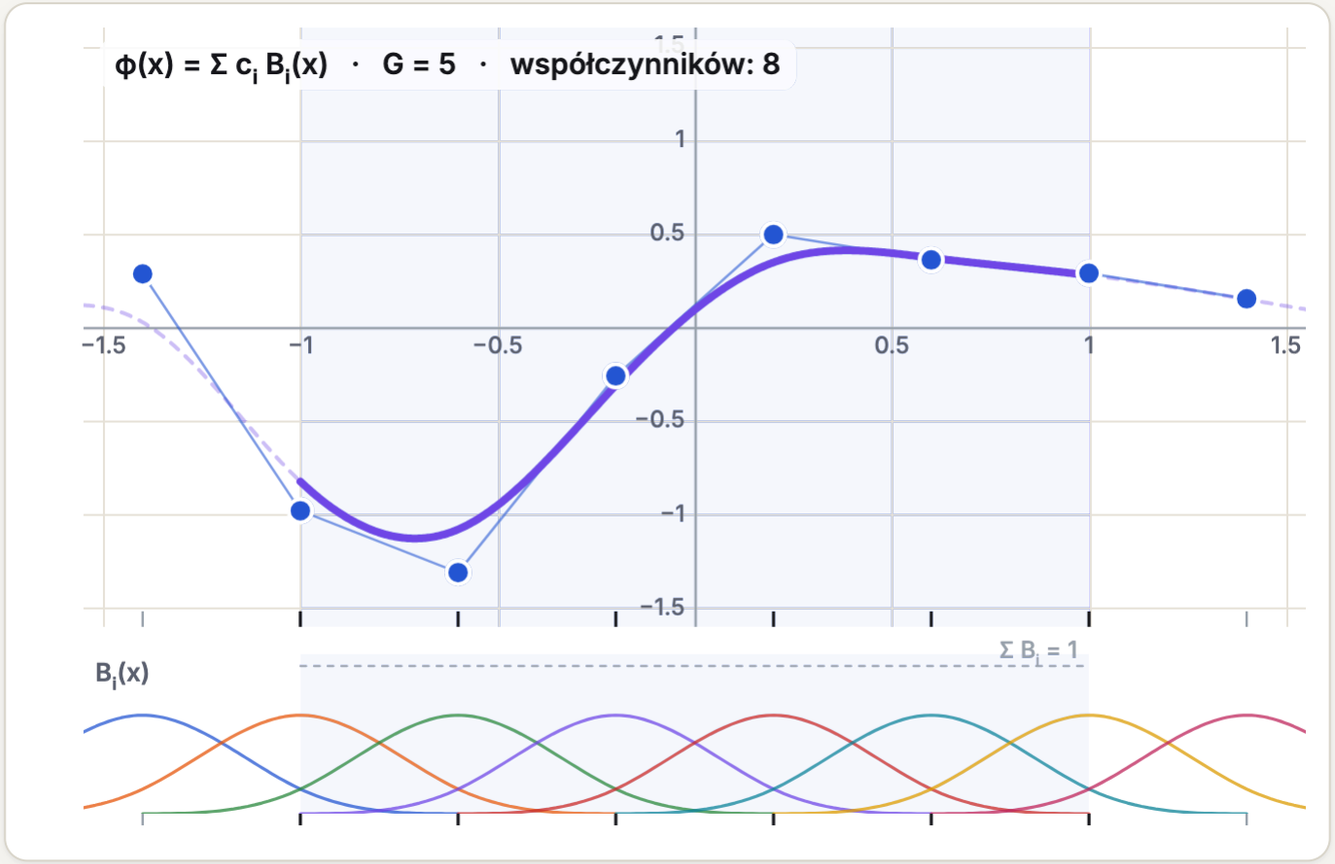
**KAN odwrotnie:** na każdej krawędzi **uczona funkcja jednej zmiennej** $\varphi$, w neuronach tylko suma.



MLP: stałe σ w węzłach


KAN: splajny na krawędziach

- Każda $\varphi$ to splajn: $\varphi(x) = \sum_i c_i B_i(x)$; uczymy współczynniki $c_i$ (Liu i in., 2024).
- Podstawa: tw. Kołmogorowa–Arnolda (1957): każdą ciągłą funkcję wielu zmiennych da się złożyć z funkcji jednej zmiennej i dodawania.
- Zaleta: wyuczone $\varphi$ można narysować i odczytać. Wada: trening wolniejszy, dokładność zwykle podobna do MLP.



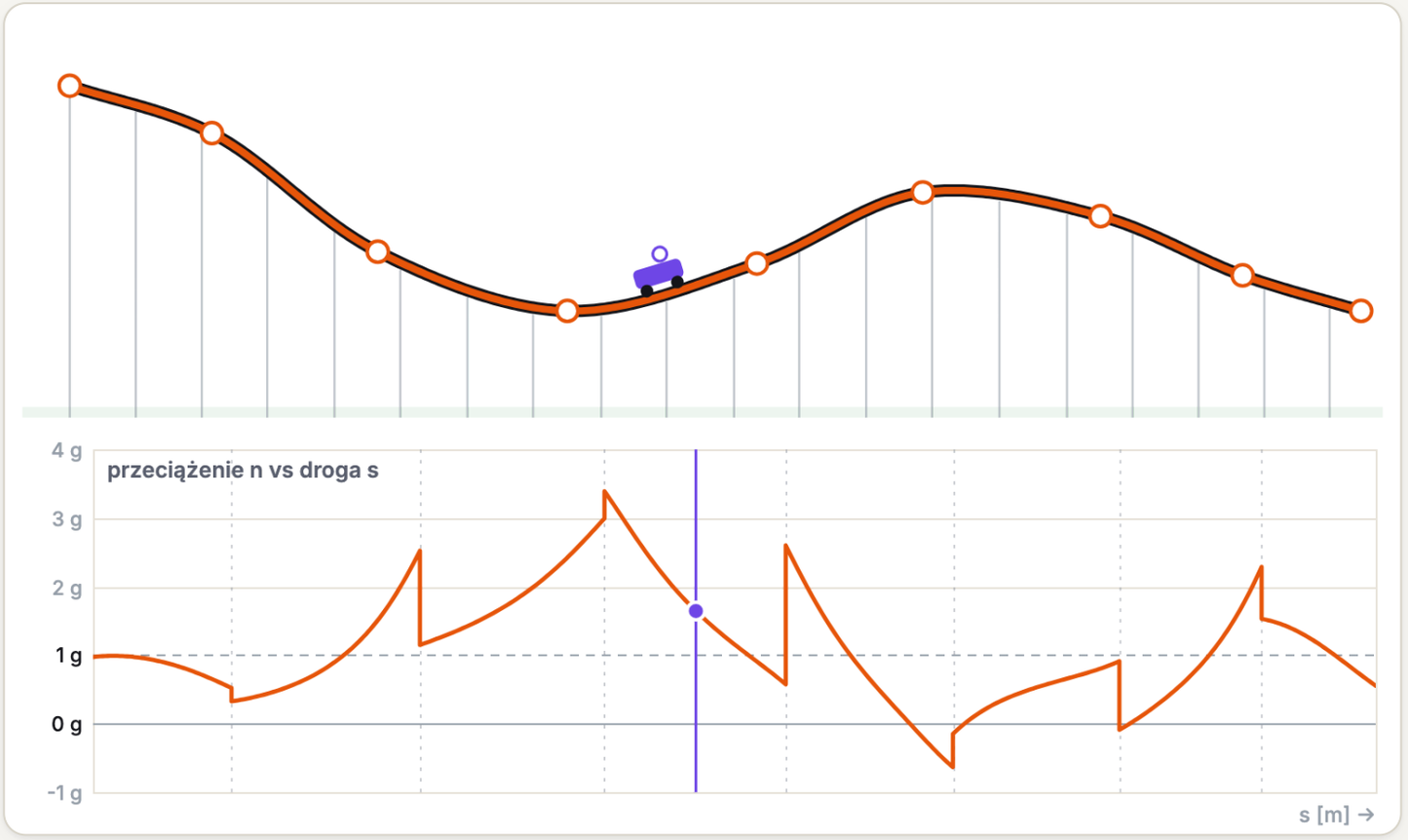
# Ciągła krzywizna: tory, drogi, kolejki górskie

ZASTOSOWANIA

Na zakręcie pasażera dociska siła proporcjonalna do **krzywizny** toru $\kappa$ (przy prędkości $v$: $a = v^2\kappa$).

- Prosta przechodzi od razu w łuk okręgu: krzywizna skacze z $0$ do $1/R$, siła pojawia się nagle: **szarpnięcie**.
- Tor z ciągłą krzywizną (splajn $C^2$): siła narasta płynnie.
- Dlatego na kolei i drogach między prostą a łukiem buduje się **krzywe przejściowe** (klotoidy), a kolejki górskie projektuje się z ciągłą krzywizną.

Demo: ten sam tor jako krzywa tylko $C^1$ i jako splajn $C^2$; na dole przeciążenie w $g$.




krzywa C¹ | splajn C² | start | n 1.64 g | v 68 km/h | max skok n w węzłach 2.03 g

# Thin-plate spline: uogólnienie na 2D

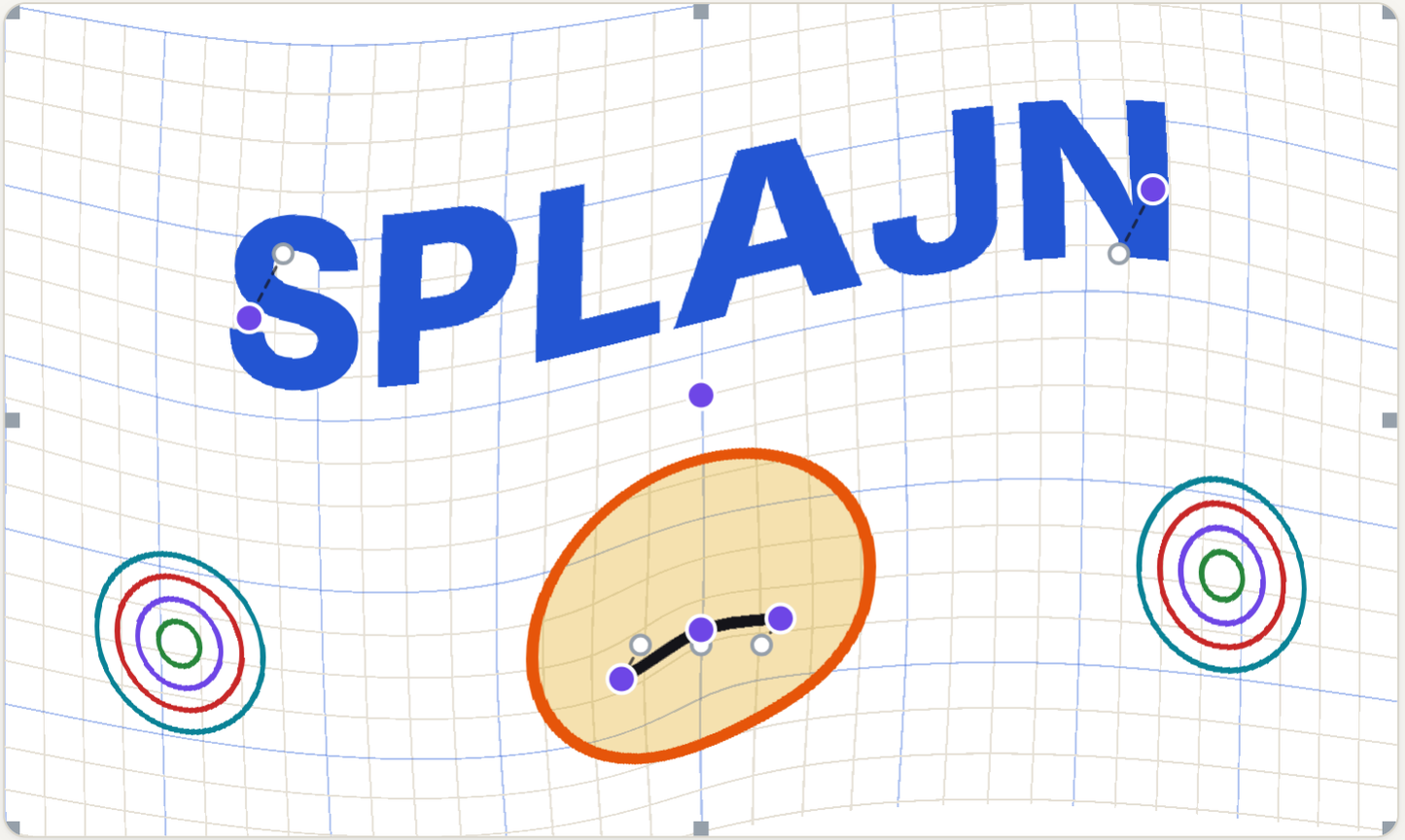
DODATEK

Cienka płyta zamiast listwy: przy warunkach $f(\mathbf{p}_i) = y_i$ minimalizujemy

$$\iint \left(f_{xx}^2 + 2f_{xy}^2 + f_{yy}^2\right) dx\, dy$$

- Rozwiązanie: $f(\mathbf{x}) = a_0 + \mathbf{a}^\top \mathbf{x} + \sum_i w_i\, U(\|\mathbf{x} - \mathbf{p}_i\|)$, $U(r) = r^2 \log r$ (Duchon, 1976).
- Morfometria i morfing (Bookstein, 1989), rejestracja obrazów medycznych, Spatial Transformer Networks.

Przeciągnij punkty: obraz wygina się jak cienka płyta.




reset | wyprostuj | losowo | energia zginania 0.0931 (j. umowne)

# Podsumowanie



Funkcja sklejana to kawałki wielomianów niskiego stopnia, połączone gładko w węzłach.

**1** **Dokładność bez oscylacji**

Chcemy gładkiej krzywej, która wiernie oddaje dane. Splajn to daje: im więcej punktów, tym dokładniej, bez oscylacji.

**2** **Szybkie obliczenia**

Jeden układ równań liniowych: zawsze jedno rozwiązanie, liczone szybko nawet dla milionów punktów.

**3** **Najmniejsza energia zginania**

Splajn naturalny to najgładsza krzywa przez dane punkty; taki kształt przyjmuje listwa kreślarska.

**4** **Zmiany lokalne**

Przesunięcie jednego punktu kontrolnego zmienia krzywą tylko w pobliżu.

Wszystkie cztery wynikają z jednego pomysłu: **wiele prostych kawałków** zamiast jednego skomplikowanego wzoru.

# Dziękuję

Pytania?

Prezentacja z interaktywnymi demami (najwygodniej na komputerze).

wojtke.github.io/funkcje-sklejane/v2/

pobierz PDF

# Literatura




### TEORIA

I. J. Schoenberg, Contributions to the problem of approximation of equidistant data by analytic functions, Quart. Appl. Math. 4 (1946), 45–99, 112–141.

J. C. Holladay, A smoothest curve approximation, Math. Tables Aids Comput. 11 (1957), 233–243.

C. A. Hall, W. W. Meyer, Optimal error bounds for cubic spline interpolation, J. Approx. Theory 16 (1976), 105–122.

M. G. Cox, The numerical evaluation of B-splines, IMA J. Appl. Math. 10 (1972), 134–149; C. de Boor, On calculating with B-splines, J. Approx. Theory 6 (1972), 50–62.

C. de Boor, A Practical Guide to Splines, Springer 1978 (wyd. popr. 2001).

Z. Fortuna, B. Macukow, J. Wąsowski, Metody numeryczne, WNT; D. Kincaid, W. Cheney, Analiza numeryczna, WNT 2006.

### GRAFIKA I CAD

G. Farin, Curves and Surfaces for CAGD, 5th ed., Morgan Kaufmann 2002; L. Piegl, W. Tiller, The NURBS Book, Springer 1997.

E. Catmull, R. Rom, A class of local interpolating splines, w: Computer Aided Geometric Design, Academic Press 1974; C. Yuksel, S. Schaefer, J. Keyser, Parameterization and applications of Catmull–Rom curves, CAD 43 (2011).

G. Chaikin, An algorithm for high-speed curve generation, CGIP 3 (1974); C. Loop, J. Blinn, Resolution independent curve rendering using programmable graphics hardware, SIGGRAPH 2005.

### STATYSTYKA I UCZENIE MASZYNOWE

C. H. Reinsch, Smoothing by spline functions, Numer. Math. 10 (1967); G. Kimeldorf, G. Wahba, A correspondence between Bayesian estimation on stochastic processes and smoothing by splines, Ann. Math. Stat. 41 (1970).

G. Wahba, Spline Models for Observational Data, SIAM 1990; P. J. Green, B. W. Silverman, Nonparametric Regression and Generalized Linear Models: A Roughness Penalty Approach, Chapman & Hall 1994; T. Hastie, R. Tibshirani, J. Friedman, The Elements of Statistical Learning, 2nd ed., Springer 2009, rozdz. 5.

F. L. Bookstein, Principal warps: thin-plate splines and the decomposition of deformations, IEEE TPAMI 11 (1989); M. Unser, Splines: a perfect fit for signal and image processing, IEEE SPM 16 (1999).

R. Balestriero, R. Baraniuk, A spline theory of deep networks, ICML 2018; Z. Liu i in., KAN: Kolmogorov–Arnold Networks, arXiv:2404.19756 (2024).